סוכן בינה מלאכותית להכנת טיוטת תביעה קטנה

תהליך האפיון

העבודה החלה בניתוח המטלה ובהגדרת מוצר ראשוני בר שימוש שניתן להשלים בתוך כשבוע: מערכת שמראיינת בעברית אדם שאינו משפטן, משלימה מידע חסר, מציגה סיכום לאישור ומפיקה טיוטת כתב תביעה הניתנת לעריכה.
נעזרתי בעוזר בינה מלאכותית כשותף לתכנון ולפיתוח. דרישות המוצר והבדיקות גובשו באופן איטרטיבי: בכל הרצה נבחנה השיחה מנקודת מבט המשתמש ונוספו תיקונים. הוחלט שכל הקליטה תתבצע בצ'אט, בשאלה ממוקדת אחת בכל תור, תוך שימוש במידע שכבר נמסר ואישור העובדות לפני הניסוח.

כלים והחלטות מרכזיות

ממשק:
Streamlit נבחר לבניית הצ'אט, עורך הטיוטה והורדת הקבצים בפייתון בלבד. חלופת FastAPI עם React מעניקה שליטה רחבה יותר במוצר ייצור, אך מגדילה את היקף הפיתוח.
תזמור ומצב:
LangGraph מנהל את שלבי החילוץ, האימות, האחזור, הניסוח והבקרה. Pydantic מגדיר את CaseData ומאמת כל מעבר. נבחר סוכן יחיד ולא Multi-Agent משום שהתהליך סדרתי ורגיש לעקביות.
מודל וחיבור:
Gemini API נבחר כדי לעמוד באילוץ העלות. LangChain Google GenAI מספק פלט מובנה ומחבר את המודל לגרף. Google Gen AI SDK הוא תלות התשתית של המתאם, ו-python-dotenv טוענת את המפתח המקומי. OpenAI ו-Claude היו חלופות אפשריות אך לא נדרשו בגרסה זו.
אחזור משפטי:
Targeted RAG מאחזר רשומות ממאגר JSON קטן, סגור ושקוף. כל רשומה מסמנת אם המקור רשמי או ציבורי ואם הטקסט הוא ציטוט או תמצית. Vector DB לא נבחר מפני שהקורפוס מצומצם וסיווג מבוסס כללים קל יותר לבקרה.
יצוא:
ReportLab יוצר את מסמכי ה-PDF ו-python-bidi מטפלת בכיוון הכתיבה. גופן Rubik מצורף לפרויקט. WeasyPrint או Chromium היו חלופות לעיצוב מורכב יותר, במחיר תלויות מערכת נוספות.

פרטיות, אימות ומקרי קצה

פרטי זיהוי וקשר נאספים בשדות מקומיים ייעודיים ומושחרים מהקשר המודל. הודעת הפתיחה נסרקת לפני קריאת מודל כדי לזהות פרטי זיהוי נפוצים. תאריכים, סכומים, תקרת תביעה, אגרה, סמכות ומקורות נבדקים בקוד דטרמיניסטי. פלט שגוי מקבל מספר ניסיונות מוגבל ולאחריהם מסלול מקומי; כל הצעת תיקון של המודל חייבת לעבור שוב את האימות המקומי.
המערכת מפיקה שני מסמכים: כתב תביעה נקי להגשה לאחר בדיקת המשתמש, ודו"ח אופציונלי נפרד המציג את המקורות ואת בקרת האיכות. זהו מוצר הדגמה לימודי ולא ייעוץ משפטי.